

高吞吐量 I/O 服务器模型设计与性能分析

系统程序设计 Lab 1

2026-05-14

高吞吐量 I/O 服务器模型设计与性能分析

摘要

高并发服务器的核心问题是如何在大量连接之间高效分配 CPU、内核 I/O 事件和用户态处理逻辑。本文围绕阻塞式 I/O、多路复用 I/O、异步 I/O 和多线程 I/O 的工作机制进行分析，并实现了三种 TCP echo server：基于 `poll()` 的单线程多路复用服务器、基于线程池的并发服务器，以及 Linux `epoll` 服务器。实验使用自编写的 `client_bench` 产生 10、100、500 个并发连接，每个连接发送 1000 个 64 字节 echo 请求，并统计吞吐量、平均延迟和 P99 延迟。实验结果表明，在 100 并发下 `epoll` 取得最高吞吐量 118200.15 req/s；在 500 并发下线程池模型达到最高吞吐量 119708.45 req/s。结果说明 I/O 模型的理论优势需要结合具体实现、负载规模和调度方式分析。最后，本文提出一种基于主从 Reactor、`epoll` 和 worker 线程池组合的高吞吐服务器架构。

关键词：UNIX I/O；`poll`；`epoll`；线程池；Reactor；高吞吐服务器

1. 引言

网络服务器需要同时处理大量客户端连接。最直接的阻塞式模型实现简单，但一个线程在一次阻塞 `read()` 或 `write()` 中等待时，无法同时处理其他连接。随着并发连接数上升，单纯阻塞模型会面临线程数量膨胀、上下文切换增加、内存消耗增加等问题。UNIX/Linux 提供了多种 I/O 并发机制：`select()`、`poll()` 和 `epoll` 属于 I/O 多路复用；POSIX AIO 与 Linux `io_uring` 代表异步 I/O 方向；线程池模型则通过多个 worker 并行处理连接。不同模型不是简单的优劣关系，而是在实现复杂度、性能、资源消耗等方面各有特点。本文的目标是：第一，梳理常见 I/O 模型的工作原理；第二，通过 echo server 实验比较 `poll`、`epoll` 和线程池模型在相同负载下的性能表现。

2. 相关工作

Stevens 的《UNIX Network Programming》系统介绍了阻塞 I/O、非阻塞 I/O 和 I/O 多路复用模型，是网络服务器编程的经典参考。对 UNIX 文件描述符、系统调用和高级 I/O 机制有更底层的讨论。Linux man-pages 给出了 `poll(2)` 和 `epoll(7)` 的接口语义。实际系统中，Nginx 采用事件驱动和 worker 进程模型，Redis 长期使用单线程事件循环处理网络 I/O，这些系统都体现了高吞吐服务器中“事件分发 + 非阻塞 I/O”的设计思想。

近年来，Linux `io_uring` 进一步降低异步 I/O 提交和完成路径上的系统调用开销。它通过共享 submission queue 和 completion queue 让用户态批量提交请求、批量收割完成事件。本文没有实现 `io_uring` 服务器，因为该项是实验可选加分项，且 `poll` 和线程池模型，并额外实现了 `epoll` 对比；但在原理分析中仍将其作为异步 I/O 的代表机制讨论。

3. I/O 模型原理分析

3.1 阻塞式 I/O

阻塞式 I/O 中，线程调用 `accept()`、`read()` 或 `write()` 后，如果条件不满足，就会进入睡眠等待。例如客户端没有发送数据时，会阻塞；发送缓冲区满时，`write()` 也可能阻塞。其优点是程序结构简单，容易理解；缺点是单线程无法同时服务大量连接。若典型流程如下：

user thread -> blocking read/write -> kernel waits for I/O -> wakeup -> user handles data

3.2 select / poll 多路复用

`select()` 和 `poll()` 允许一个线程同时等待多个 `fd` 的事件。应用把一组 `fd` 交给内核，内核在其中任意 `fd` 就绪时返回，用户态再检查哪些 `fd` 可读或可写。

`poll()` 相比 `select()` 没有固定的 `FD_SETSIZE` 位图限制，接口也更直接。但 `poll()` 每次调用仍需要传入整个 `pollfd` 数组，返回后应用通常需要线性扫描数组来找就绪 `fd`。因此当总连接数很大、活跃连接较少时，`poll()` 的 $O(n)$ 扫描成本会越来越明显。

3.3 epoll

`epoll` 是 Linux 提供的可扩展 I/O 多路复用机制。它把“注册 `fd`”和“等待事件”拆开：应用先通过 `epoll_ctl()` 把 `fd` 加入内核维护的 interest list，再通过 `epoll_wait()` 获取 ready list 中的活跃事件。相对于 `poll()` 每次传入整个 `fd` 数组，`epoll` 更适合大量连接、少量活跃的场景。

`epoll` 有水平触发（LT）和边缘触发（ET）两种常见模式。LT 模式下，只要 `fd` 仍处于可读或可写状态，`epoll_wait()` 就可能持续返回该事件，编程模型较安全。ET 模式只在状态变化时通知一次，减少重复通知，但要求应用使用非阻塞 `fd`，并循环读写直到 `EAGAIN`，否则可能遗漏数据。本实验的 `epoll_server` 使用 LT 思路，优先保证实现稳定。

3.4 异步 I/O

异步 I/O 的目标是让应用提交 I/O 请求后不阻塞等待，等内核完成后再通过信号、回调或完成队列通知用户态。POSIX AIO 提供 `aio_read()`、`aio_write()` 等接口，但在 Linux 网络 I/O 实践中使用并不广泛。Linux `io_uring` 通过共享环形队列提交和完成请求，减少系统调用次数并支持批量操作，在高性能存储和网络场景中有很强潜力。

多路复用和异步 I/O 的本质区别在于：多路复用通常通知“`fd` 已就绪”，应用随后自己执行 `read/write`；异步 I/O 则提交“具体 I/O 操作”，内核完成后通知“操作已完成”。

3.5 多线程 I/O

多线程模型通过多个 `worker` 并发处理连接或任务。线程池避免了每次请求都创建线程的开销，也能利用多核 CPU。代价是需要任务队列、互斥锁和条件变量，可能出现锁竞争、队列等待和上下文切换。对于 `echo` 这类简单 I/O 任务，线程池模型可能因为并行处理多个连接而取得不错吞吐；但在极大连接数下，线程数量和阻塞 I/O 仍然需要谨慎控制。

4. 实验设计与实现

4.1 实验环境

项目	信息
操作系统	Ubuntu 20.04.6 LTS
内核	Linux rdma221 5.4.0-26-generic #30-Ubuntu SMP Mon Apr 20 16:58:30 UTC 2020 x86_64 x86_64 x86_64 GNU/Linux
CPU	Intel(R) Xeon(R) Silver 4314 CPU @ 2.40GHz, 64 logical CPUs
内存	Mem: 251Gi 110Gi 134Gi 2.0Mi 6.0Gi 138Gi
文件系统	/dev/mapper/ubuntu--vg-ubuntu--lv ext4 1880200276 1225559920 568832300 69% /
存储	SSD/NVMe, including Dell DC NVMe PE8010 RI U.2 960GB

4.2 程序实现

本文实现了三个 TCP echo server：

- `poll_server`：单线程 `poll()` 事件循环，维护 `pollfd` 数组，监听连接和客户端读事件。
- `threadpool_server`：主线程负责 `accept()`，固定数量 `worker` 线程从队列取连接并使用阻塞 I/O 处理 `echo`。

- `epoll_server`: Linux-only, 监听 fd 和客户端 fd 均使用非阻塞模式, 并通过 `epoll_wait()` 获取活跃事件。

压测程序 `client_bench` 使用多个客户端线程, 每个线程维护一个 TCP 连接, 循环发送固定长度请求并等待 echo 响应。每个请求记录一次延迟, 最终输出总吞吐量、平均延迟、P50、P95 和 P99。

4.3 实验参数

```
message size           = 64 bytes
connections            = 10, 100, 500
requests per connection = 1000
total requests         = connections * 1000
poll_server port       = 8080
threadpool_server port = 8081
epoll_server port      = 8082
threadpool workers     = 4
```

实验运行命令记录在 `docs/EXPERIMENT_FLOW.md` 中。

5. 实验结果与分析

5.1 原始结果表

server	connections	msg_size	total_requests	total_time_sec	throughput_req_avg_ms	p50_ms	p95_ms	p99_ms
epoll	10	64	10000	0.123677	80856.10	0.107355	0.065374	0.265751
poll	10	64	10000	0.125117	79925.39	0.105455	0.060416	0.269866
threadpool	10	64	10000	0.145204	68868.60	0.107069	0.034614	0.270230
epoll	100	64	100000	0.846023	118200.15	0.757371	0.752847	1.458624
poll	100	64	100000	0.867852	115226.97	0.706775	0.737443	1.184062
threadpool	100	64	100000	0.874225	114387.08	0.440331	0.033368	0.044575
epoll	500	64	500000	4.961944	100766.96	4.850345	5.243275	5.670742
poll	500	64	500000	4.422673	113053.79	4.186389	4.269276	5.057813
threadpool	500	64	500000	4.176814	119708.45	2.036434	0.032242	0.040961

5.2 吞吐量

吞吐量结果显示, 10 并发时 `epoll` 和 `poll` 接近, 分别为 80856.10 req/s 与 79925.39 req/s, 线程池为 68868.60 req/s。100 并发时 `epoll` 达到 118200.15 req/s, 是该并发下最高值。500 并发时线程池达到 119708.45 req/s, 是本实验中的最高吞吐量为 113053.79 req/s, `epoll` 为 100766.96 req/s。

这个结果说明, 理论上更可扩展的 I/O 模型并不必然在所有实验规模下胜出。本实验中的 echo 逻辑非常轻量, 且 `epoll_server` 是单线程实现, 所有连接的读取和写回都在一个事件循环中完成; 线程池模型则可以让多个 worker 同时处理连接, 在 500 并发下获得了更高吞吐。

5.3 平均延迟

平均延迟随连接数增加整体上升。`poll` 在 500 并发下平均延迟为 4.186389 ms, `epoll` 为 4.850345 ms, 线程池为 2.036434 ms。线程池在高并发下平均延迟较低, 说明多个 worker 并行处理 echo 请求降低了单事件循环排队压力。

5.4 P99 延迟

P99 反映尾部延迟。`poll` 在 500 并发下 P99 达到 7.717189 ms, `epoll` 为 5.959882 ms, 而线程池为 0.049261 ms。需要注意的是, 线程池数据中平均延迟高于 P99, 这是因为每个 worker 以连接为单位处理请求, 少数连接可能在进入 worker 前经历较长队列等待, 这些极端等待拉高平均值; 但如果它们占全部请求的比例不足 1%, 就不会体现在 P99 中。

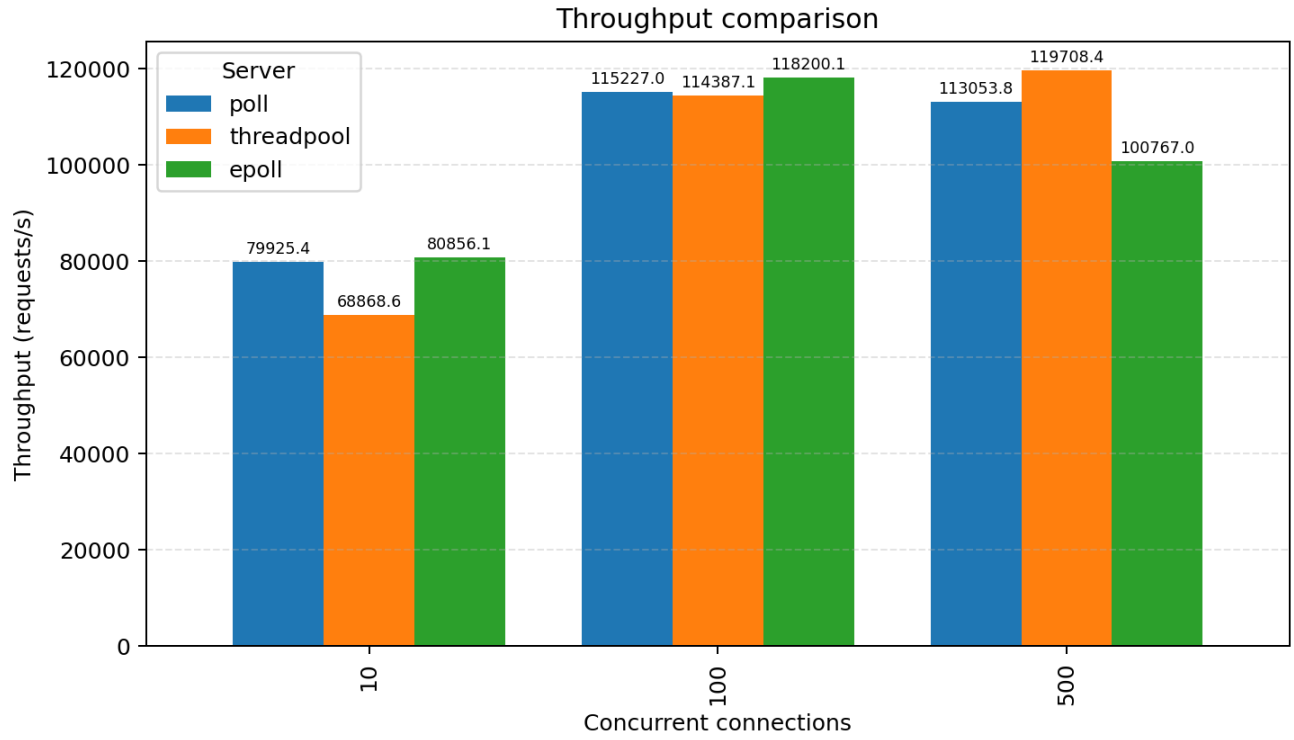


Figure 1: Throughput comparison

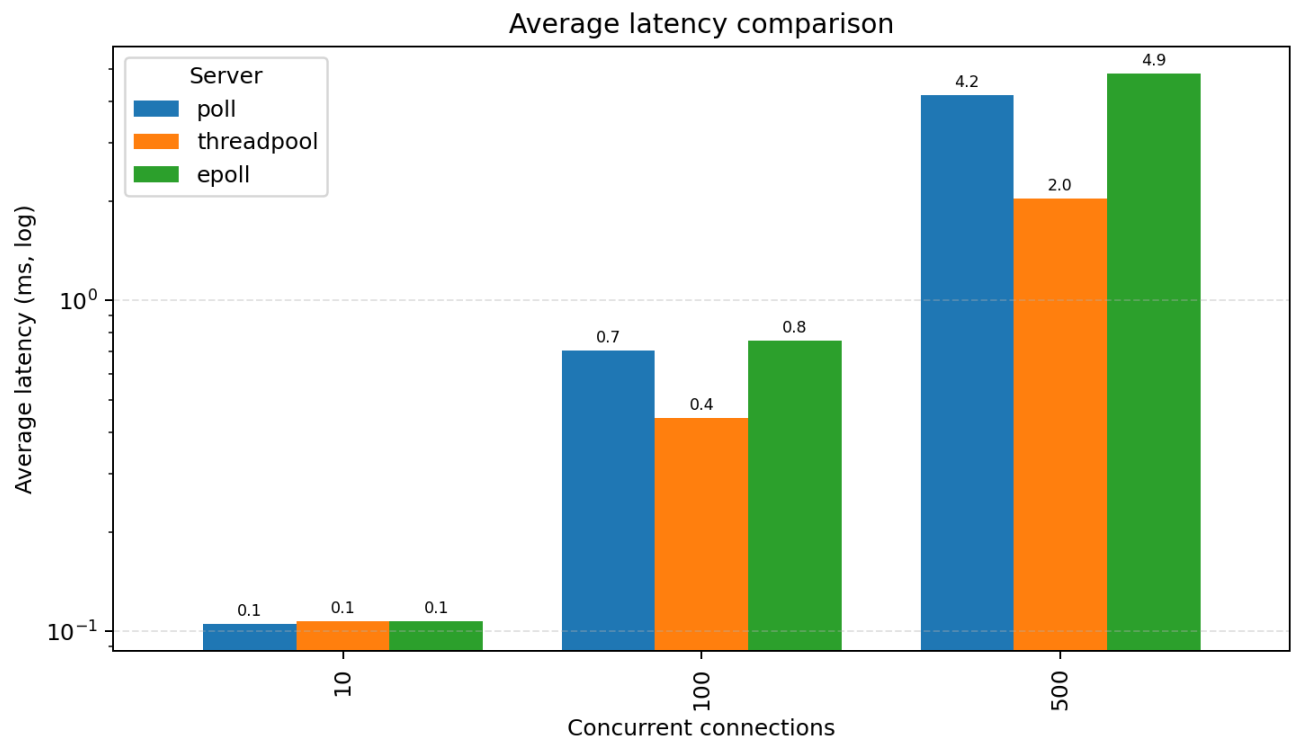


Figure 2: Average latency comparison

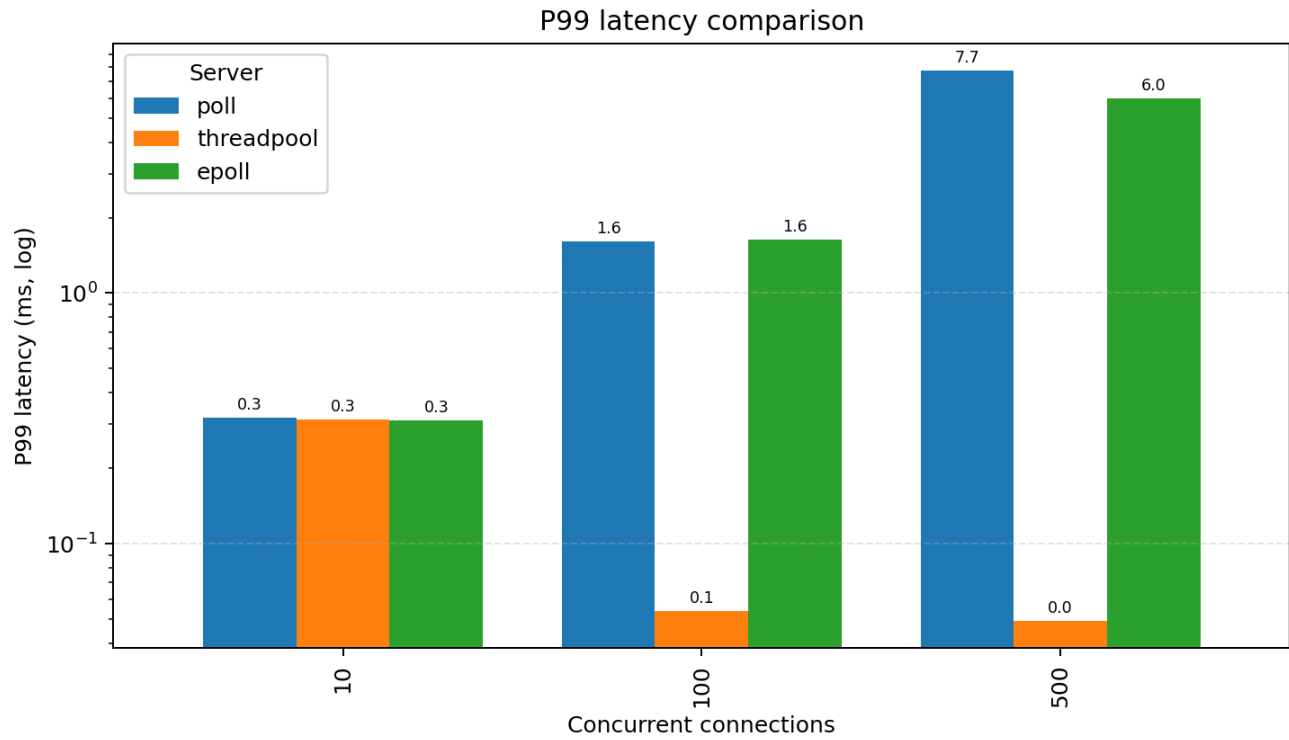


Figure 3: P99 latency comparison

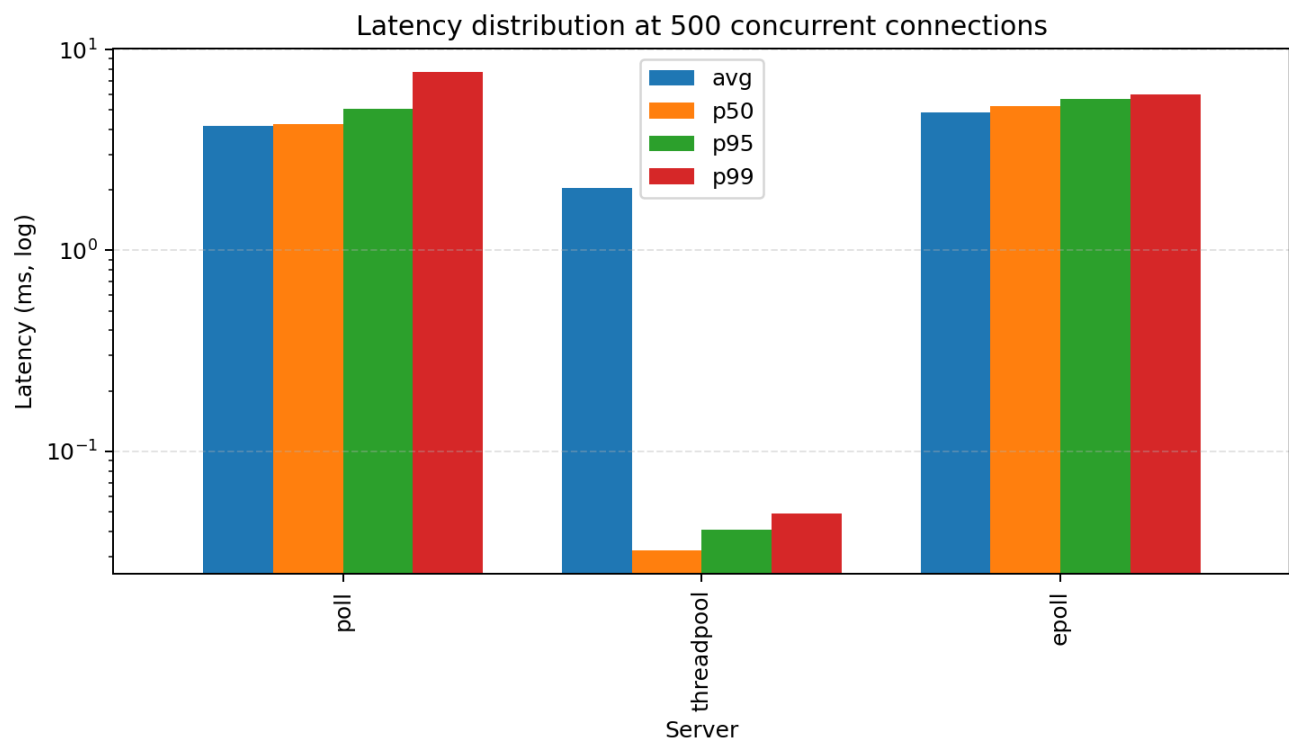


Figure 4: Latency distribution at 500 connections

5.5 结果讨论

`poll` 的实现简单，但每轮需要扫描 `fd` 数组。连接数从 100 增加到 500 后，吞吐量没有明显增长，平均延迟和 P99 都上升，符合线性扫描成本和单线程处理能力受限的预期。

`epoll` 在 100 并发下吞吐量最高，体现了只返回活跃事件的优势。但在 500 并发下低于 `poll` 和线程池，可能与本实验实现有关：请求很小，事件分发收益可能被实现细节、调度和缓存效应抵消。

线程池模型在 500 并发下吞吐量最高，说明在当前机器 64 logical CPUs 的环境中，即使只使用 4 个 worker，也能通过并行连接处理

6. 高吞吐量服务器架构设计

基于以上分析，本文推荐使用“主从 Reactor + `epoll` + worker thread pool”的组合架构。

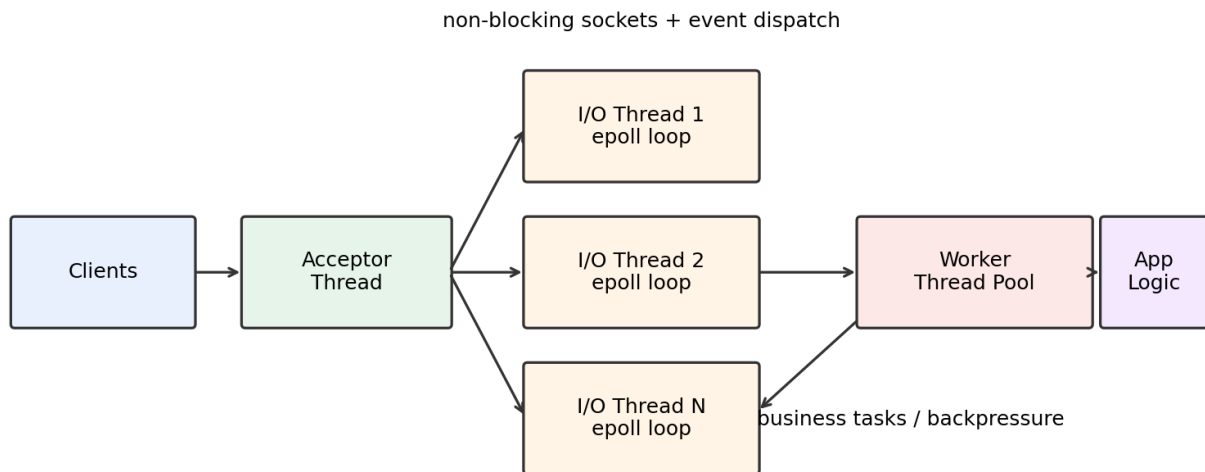


Figure 5: Recommended high-throughput server architecture

设计要点如下：

1. acceptor 线程只负责监听端口和接收连接，避免复杂业务阻塞新连接接入。
2. 多个 I/O 线程各自维护一个 `epoll` 实例，连接按 `fd`、哈希或负载均衡策略分配到不同 I/O 线程。
3. I/O 线程使用非阻塞 `socket`，只负责事件分发、协议解析和轻量级读写。
4. CPU 密集或可能阻塞的业务逻辑交给 worker 线程池处理。
5. 每个连接维护输入/输出缓冲区；当发送缓冲区积压过多时启用 `backpressure`，避免慢客户端耗尽内存。
6. 对于大规模部署，可以使用多进程或多实例绑定 CPU/NUMA 节点，减少跨 NUMA 访问和锁竞争。

这种架构融合了 `epoll` 处理大量连接的能力和线程池利用多核 CPU 的能力。它也解释了实验结果中的现象：单线程事件循环适合 worker 并行化来保持吞吐。

7. 局限性

本实验仍有一些限制：

- `echo` 协议过于简单，不能完全代表真实业务服务器。
- 每种并发规模只记录一次结果，没有多次重复取平均值。
- 测试发生在本机回环地址 `127.0.0.1`，不包含真实网络延迟和丢包。
- 本文比较了 10、100、500 三档并发连接，但没有继续扫描服务器可承受的绝对最大连接数。

- 本文没有单独采集 CPU 使用率和内存占用曲线，后续可使用 `pidstat`、`top`、`perf` 等工具补充。
- `epoll_server` 使用简化 LT 模式实现，没有进一步实现 ET 模式、输出缓冲队列和多 Reactor。
- 本文没有实现 `io_uring_server`，仅在原理层面对异步 I/O 进行讨论。

这些限制不影响本实验对不同 I/O 模型趋势的观察，但在正式生产级性能评估中需要更完整的负载模型和多轮统计。

8. 结论

本文分析并实现了多种高并发 I/O 服务器模型。实验表明，`poll` 简单直接，但在连接数增加时会受到线性扫描和单线程处理能力限制；在 100 并发下取得最高吞吐，但简化单线程实现并不保证所有场景下优于其他模型；线程池模型在 500 并发下取得最高吞吐，说明 `echo` 负载有效。

综合原理和实验结果，高吞吐服务器不应机械选择单一模型。更合理的工程方案是使用 `epoll` 或类似事件机制进行连接事件分发，配合 `worker` 线程池，从而在大量连接管理和多核计算之间取得平衡。

参考文献

- [1] W. Richard Stevens, Stephen A. Rago. *Advanced Programming in the UNIX Environment*. Addison-Wesley.
- [2] W. Richard Stevens, Bill Fenner, Andrew M. Rudoff. *UNIX Network Programming, Volume 1: The Sockets Networking API*. Addison-Wesley.
- [3] Linux man-pages project. `poll(2)` and `epoll(7)` manual pages.
- [4] Nginx Documentation. Nginx architecture and event processing model.
- [5] Redis Documentation. Redis event loop and networking implementation notes.